

Fig 1. Configuration of the RIPS scan

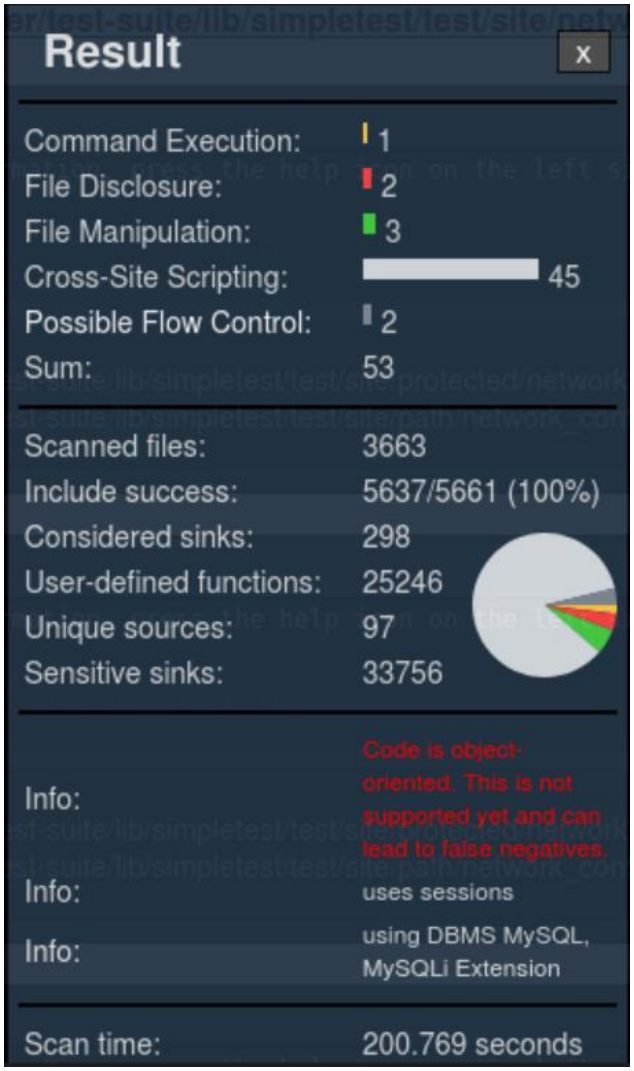


Fig 2. Result of the RIPS scan

The process of setting up RIPS was easy. The hard part was getting it to run at the beginning. The way I have Hackazon configured it runs on the local host domain. Any extension of local host, such as localhost/RIPS, takes you to a Hackazon page instead. To combat this, I needed to configure RIPS to run on localhost:8080. As can be seen from figure 2, there are a lot of XSS vulnerabilities in Hackazon. Let's look at and analyze three XSS vulnerabilities on the site. Screenshots of all the vulnerabilities can be found at the bottom of the paper. Let's look at three of the vulnerabilities the RIPS scan found:

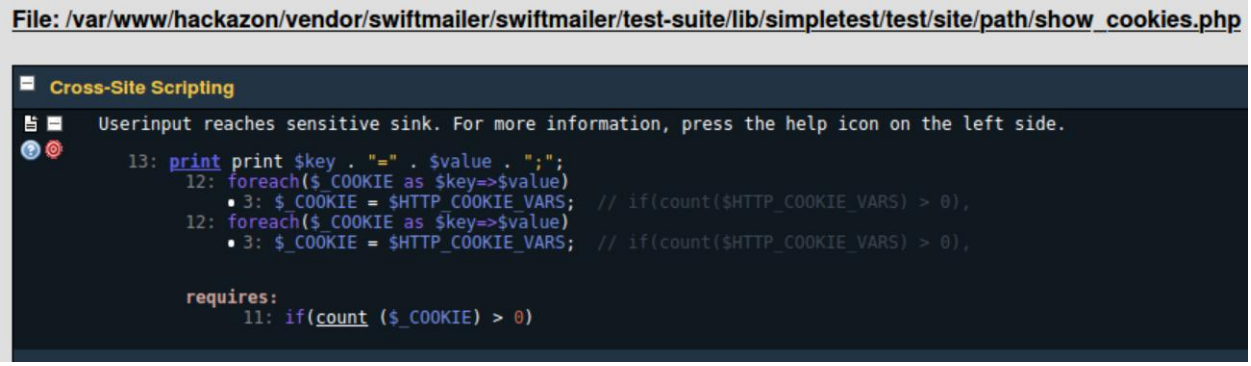


Fig 1. Vulnerability one

This section of code is getting information from the cookies in the browser. However, the data of the cookies can be modified in the browser, and the input is not being sanitized. This means that someone can edit the value of a cookie to run a script on Hackazon. This can allow the attacker to steal session data or other information from the user.

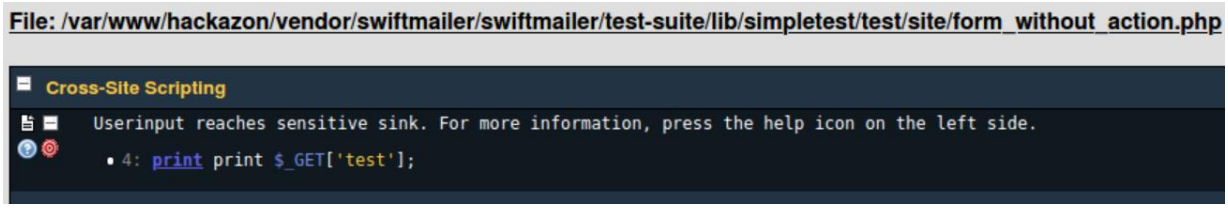


Fig 2. Vulnerability two

This vulnerability is a reflected XSS vulnerability. The value of test comes from a different URL, http://example.com/page.php?test=value, which can be maliciously changed to something like: http://localhost/page.php?test=<script>alert('XSS')</script>. This could allow the attacker to do something like send a malicious link to a company. Anyone who clicks on the link is redirected to an official website, but their session information is stolen by the attacker because of the script extension to the URL.

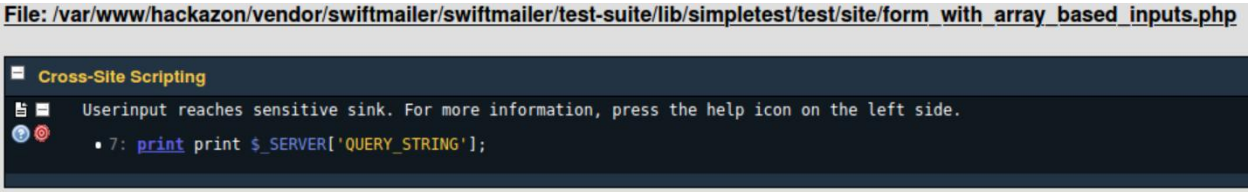


Fig 3. Vulnerability three

Like the previous vulnerability the attacker can put anything into the URL. Specifically, anything after the question mark in the link address, http://localhost/page.php?name=John. The attacker could put a script into the link, http://localhost/page.php?<script>alert('XSS')</script>, that allows them to steal information from anyone who uses the link.

XSS vulnerabilities are a type of injection attack. In these attacks, malicious scripts are injected into trusted websites. In XSS attack these scripts are sent to another user, and their browser runs them, thinking they are from the trusted website. There are two main types of XSS attacks. These two types are reflected and stored (Cross Site Scripting (XSS) | OWASP Foundation, n.d.). Reflected XSS are delivered to the victim via some external source. This could be email, malicious links or any other way of getting a victim to a website. As we saw in the second and third vulneraries, the URLs can be changed to add a malicious script. A stored XSS is what happens when an XSS script is stored on the servers of the website. The server itself sends the script to anyone who visits the site. The absolute worst-case scenario would be a large, well

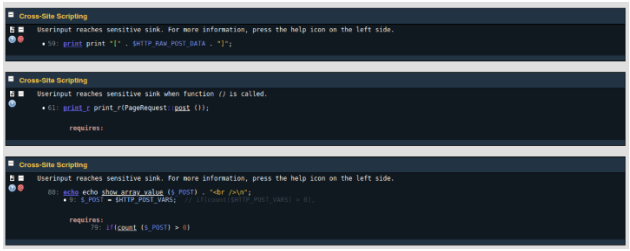
known and much-used website getting infected with a stored XSS. If Amazon got infected with a stored XSS every single person who goes to the site would get their browser infected. Large amounts of data would be available to the attacker. The only way to stop the XSS infection would be to go through the database until it is located and removed. These vulnerabilities allow malicious actors to take information from normal users.

References:

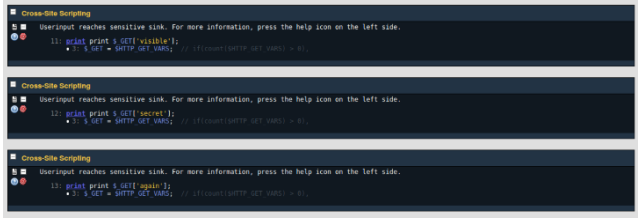
Cross Site Scripting (XSS) | OWASP Foundation. (n.d.). https://owasp.org/www-community/attacks/xss/

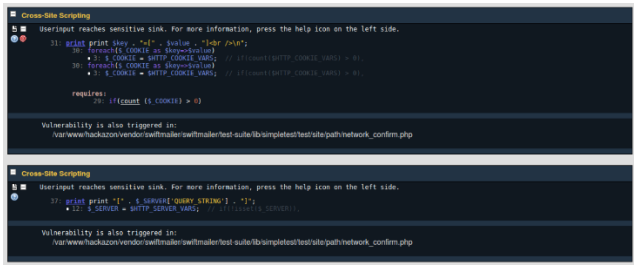




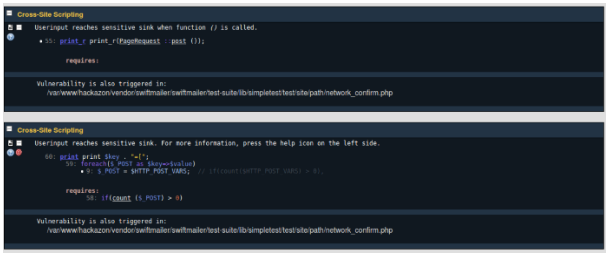


File: /var/www/hackazon/vendor/swiftmailer/swiftmailer/test-suite/lib/simpletest/test/site/self_form.php





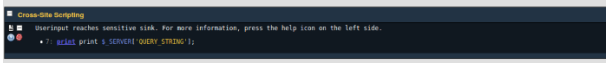








File: /var/www/hackazon/vendor/swiftmailer/swiftmailer/test-suite/lib/simpletest/test/site/form_with_array_based_inputs.php



hide all

File: /var/www/hackazon/vendor/swiftmailer/swiftmailer/test-suite/lib/simpletest/test/site/upload_handler.php







File: /var/www/hackazon/vendor/swiftmailer/swiftmailer/test-suite/lib/simpletest/test/site/form_with_quoted_values.php



hide all

File: /var/www/hackazon/vendor/swiftmailer/swiftmailer/test-suite/lib/simpletest/test/site/path/show_cookies.php



File: /var/www/hackazon/vendor/swiftmailer/swiftmailer/test-suite/lib/simpletest/test/site/form_without_action.php





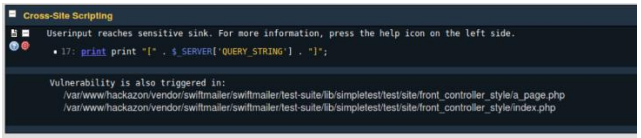
hide all

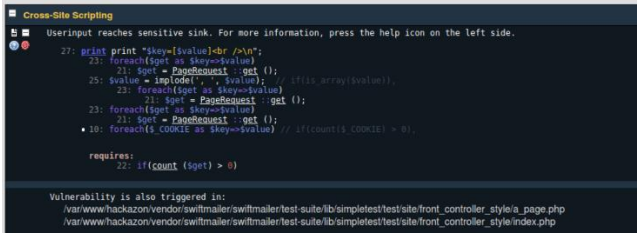
File: /var/www/hackazon/vendor/swiftmailer/swiftmailer/test-suite/lib/simpletest/test/site/front_controller_style/show_request.php

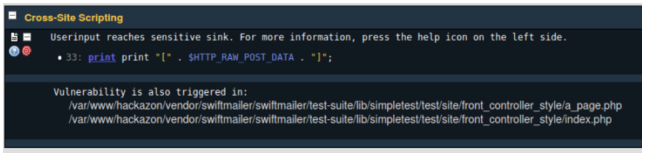


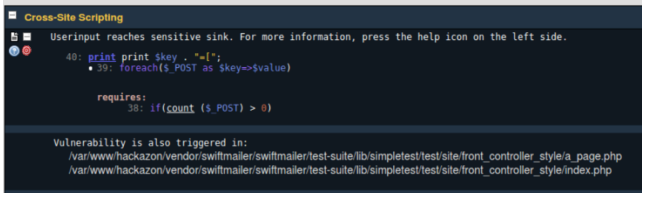




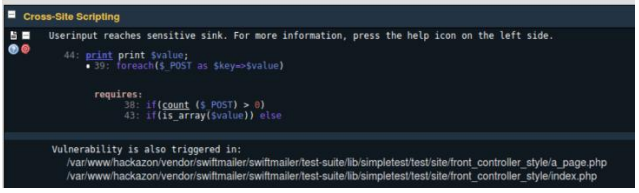






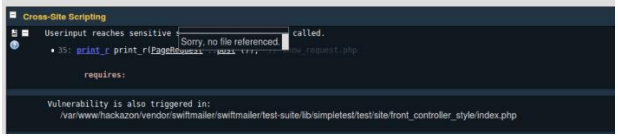






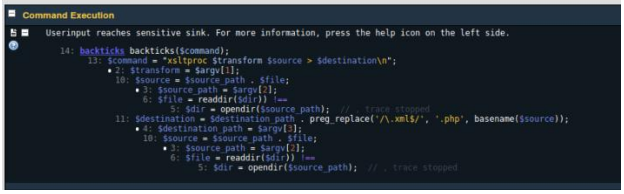
File: /var/www/hackazon/vendor/swiftmailer/swiftmailer/test-suite/lib/simpletest/test/site/front_controller_style/a_page.php





File: /var/www/hackazon/vendor/swiftmailer/swiftmailer/test-suite/lib/simpletest/packages/transform_all_lastcraft.php





File: /var/www/hackazon/vendor/guzzlehttp/guzzle/tests/ClientTest.php





File: /var/www/hackazon/assets/views/cart/view.php



hide all

File: /var/www/hackazon/assets/views/common/start.php



File: /var/www/hackazon/web/amf_back_office/Profiler.php

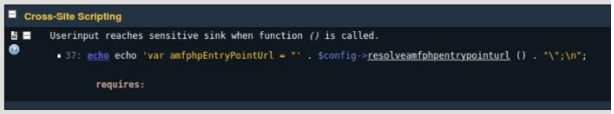


hide all

File: /var/www/hackazon/web/amf_back_office/SignIn.php



File: /var/www/hackazon/web/amf_back_office/index.php



hide all

File: /var/www/hackazon/web/amf_back_office/ClientGenerator.php

